

TECNOLOGÍAS DE LA INFORMACIÓN

Desarrollo de Sistemas Informáticos

4 créditos



Profesores:

Ing. Roly Steeven Cedeño Menéndez, Mtr.

Titulaciones	Semestre
• <i>Tecnologías de la Información en Línea</i>	<i>Quinto</i>

NOMBRES Y APELLIDOS: CONGO ARROYO LUIS ULPIANO

PARALELO: "A"

SEMESTRE: QUINTO

ACTIVIDAD: # 8 RESOLUCIÓN DE EJERCICIOS

PERIODO ABRIL 2026



Actividades a desarrollar en la Unidad 4

Actividad # 8

Objetivo de la tarea

Construir la arquitectura del lado del servidor (Backend) y la base de datos del Sistema de Gestión de Incidentes, aplicando tecnologías de nivel industrial (Node.js/Express o Java/Spring Boot con MongoDB/JPA) para exponer una API RESTful escalable y segura.

INTRODUCCIÓN

En el presente trabajo se desarrolla una Interfaz de Programación de Aplicaciones (API REST) para la gestión de tickets de soporte técnico, utilizando tecnologías estándar como Java Spring Boot y PostgreSQL. El objetivo es contar con un sistema funcional que permita registrar y administrar incidentes de manera ordenada y eficiente.

OBJETIVO

Desarrollar una API estructurada siguiendo buenas prácticas de programación.

- *Implementar operaciones completas de Crear, Leer, Actualizar y Eliminar (CRUD).*
- *Conectar y sincronizar correctamente la aplicación con la base de datos PostgreSQL.*
- *Validar el funcionamiento de todos los servicios mediante pruebas controladas.*
- *Preparar el proyecto para su almacenamiento en repositorio y despliegue en la nube.*

TECNOLOGÍAS UTILIZADAS

Lenguaje de programación: Java 17

- *Framework: Spring Boot 3.3.2*
- *Gestor de dependencias: Maven*
- *Base de datos: PostgreSQL*
- *Editor de código: Visual Studio Code*
- *Pruebas de API: Postman*
- *Control de versiones: Git / GitHub*
- *Plataforma de despliegue: Render*



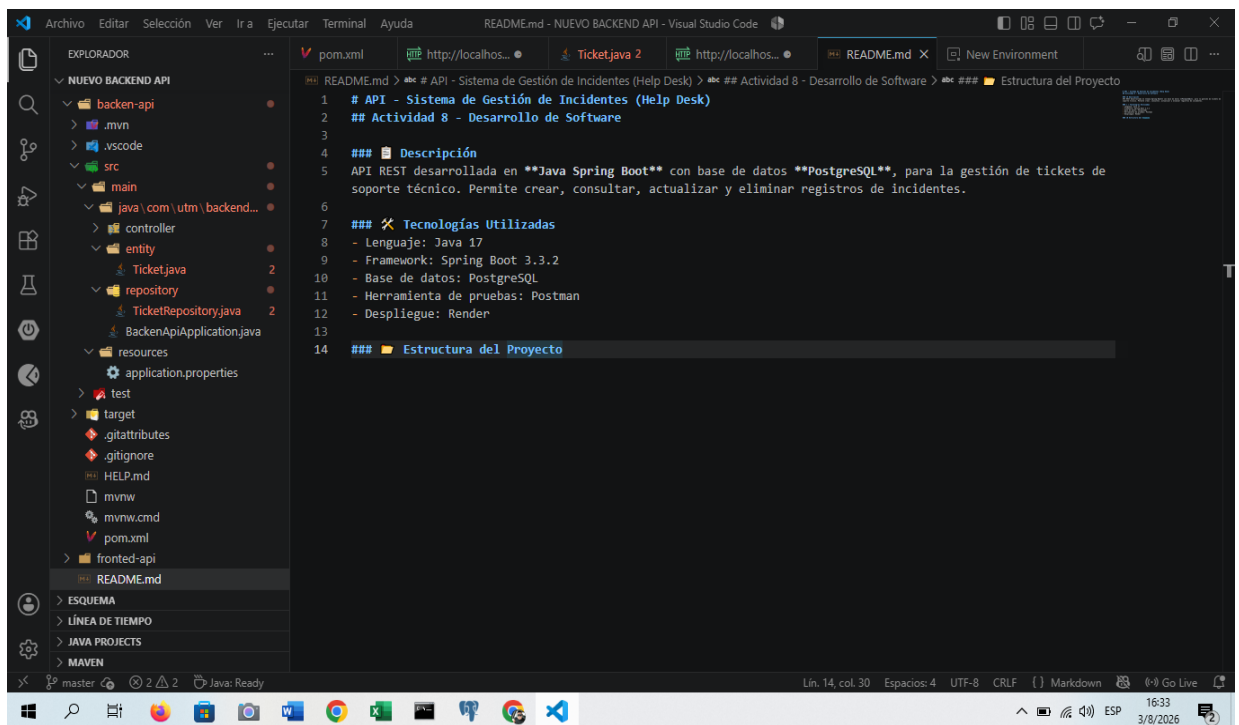
ESTRUCTURA DEL PROYECTO

NUEVO BACKEND API/

```
└─ backen-api/  
  └─ src/  
    └─ main/  
      └─ java/com/utm/backend_api/  
        └─ controller/  
          └─ entity/  
            └─ repository/  
              └─ BackenApiApplication.java  
            └─ resources/  
              └─ application.properties  
          └─ pom.xml
```

|___Fronted-api

|___Readme.md

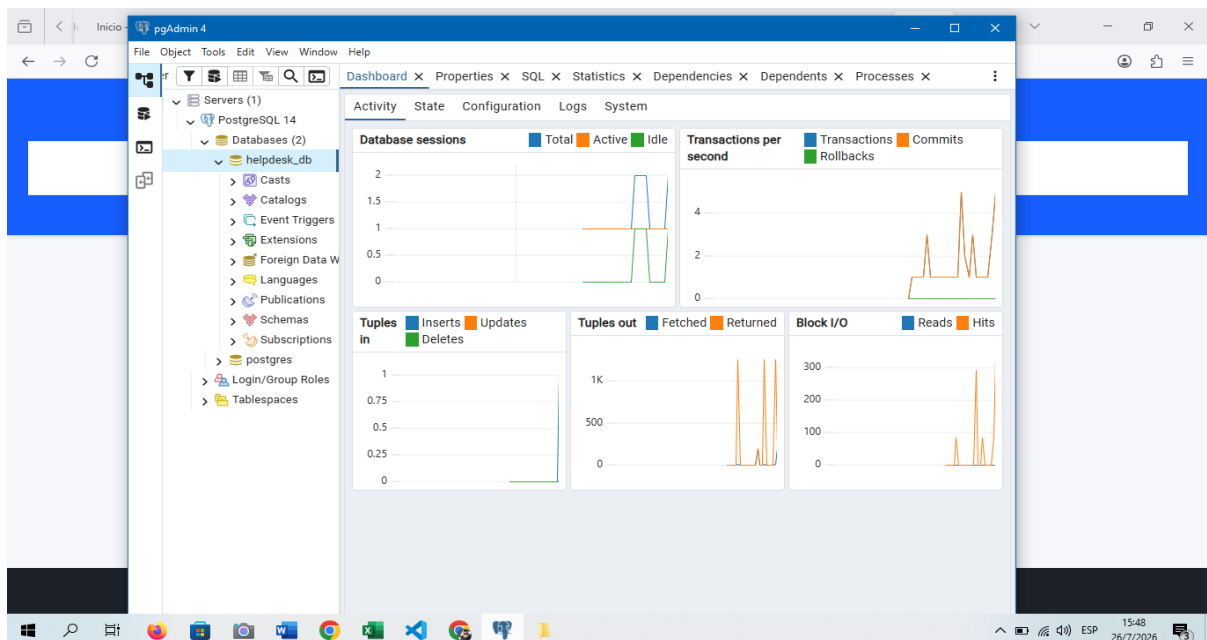




DESARROLLO Y CONFIGURACIÓN

Base de Datos

- Se instaló y configuró PostgreSQL
- Se creó la base de datos con el nombre: *helpdesk_db*
- Se definió la estructura de la tabla *tickets* con los campos:
- *id* : Identificador único automático
- *titulo* : Resumen del incidente
- *categoria* : Área afectada (Red / Hardware / Software)
- *prioridad* : Nivel de urgencia (Alta / Media / Baja)
- *descripcion* : Detalle completo del problema
- *estado* : Situación del ticket (Abierto / En Progreso / Resuelto)
- *createdAt* y *updatedAt* : Fechas automáticas de registro y modificación





API REST Implementada

Se desarrollaron todas las rutas para cumplir el CRUD completo:

OPERACIÓN RUTA FUNCIÓN

Consultar todos `GET /tickets` Obtiene el listado completo de incidentes

Consultar uno `GET /tickets/:id` Muestra los datos de un solo ticket

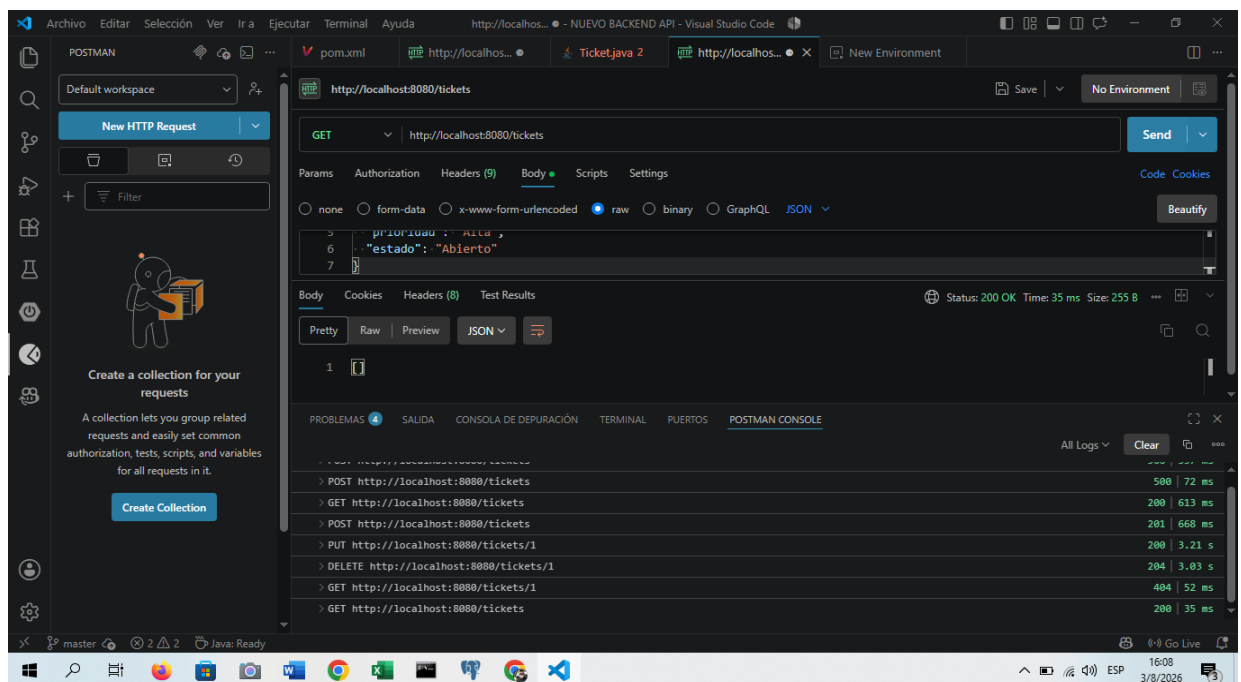
Registrar `POST /tickets` Guarda un nuevo incidente en la base de datos

Actualizar `PUT /tickets/:id` Modifica datos o cambia el estado de un ticket

Eliminar `DELETE /tickets/:id` Borra permanentemente un registro

Evidencias

GET





POST

POSTMAN

Default workspace

New HTTP Request

Filter

Create a collection for your requests

A collection lets you group related requests and easily set common authorization, tests, scripts, and variables for all requests in it.

Create Collection

POST http://localhost:8080/tickets

Params Authorization Headers (9) Body Scripts Settings

Body

```
{  "prioridad": "Alta",  "estado": "Abierto"}
```

Body Cookies Headers (9) Test Results

Pretty Raw Preview JSON

Status: 201 Created Time: 67 ms Size: 475 B

PROBLEMAS SALIDA CONSOLA DE DEPURACIÓN TERMINAL PUERTOS POSTMAN CONSOLE

All Logs Clear

Request	Response
GET http://localhost:8080/tickets	200 613 ms
POST http://localhost:8080/tickets	201 668 ms
PUT http://localhost:8080/tickets/1	200 3.21 s
DELETE http://localhost:8080/tickets/1	204 3.03 s
GET http://localhost:8080/tickets/1	404 52 ms
GET http://localhost:8080/tickets	200 35 ms
POST http://localhost:8080/tickets	201 67 ms

PUT

POSTMAN

Default workspace

New HTTP Request

Filter

Create a collection for your requests

A collection lets you group related requests and easily set common authorization, tests, scripts, and variables for all requests in it.

Create Collection

PUT http://localhost:8080/tickets/1

Params Authorization Headers (9) Body Scripts Settings

Body

```
{  "id": 1,  "titulo": "Falla de conexión a red",  "descripcion": "No tengo acceso a internet en mi equipo",  "categoria": "Red",  "prioridad": "Alta",  "estado": "Abierto"}
```

Body Cookies Headers (8) Test Results

Pretty Raw Preview JSON

Status: 200 OK Time: 3.21 s Size: 409 B

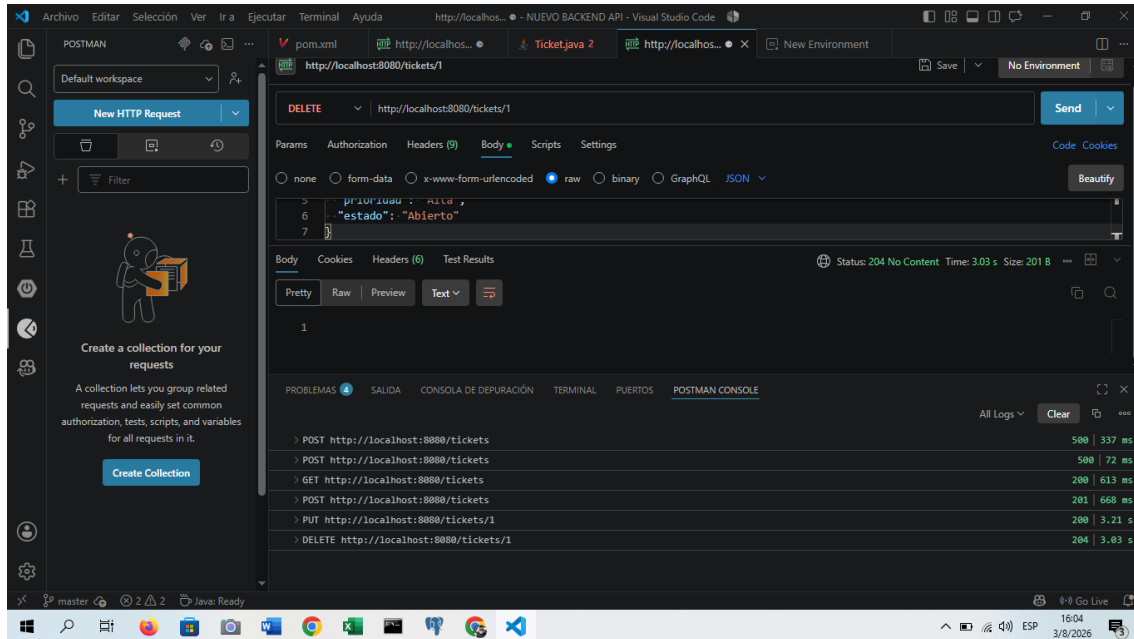
PROBLEMAS SALIDA CONSOLA DE DEPURACIÓN TERMINAL PUERTOS POSTMAN CONSOLE

All Logs Clear

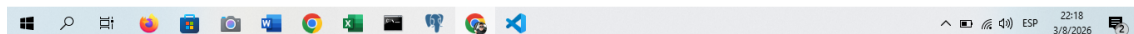
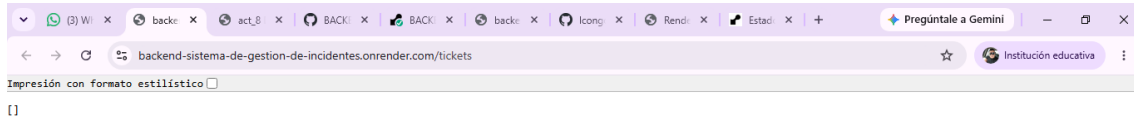
Request	Response
POST http://localhost:8080/tickets	500 337 ms
POST http://localhost:8080/tickets	500 72 ms
GET http://localhost:8080/tickets	200 613 ms
POST http://localhost:8080/tickets	201 668 ms
PUT http://localhost:8080/tickets/1	200 3.21 s

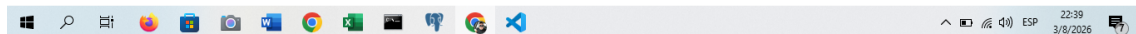
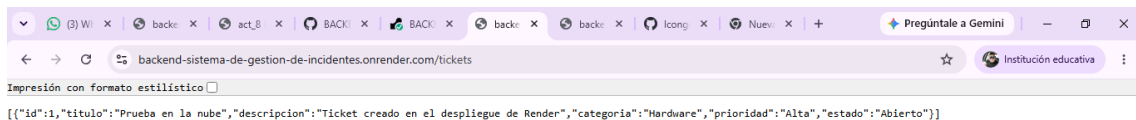
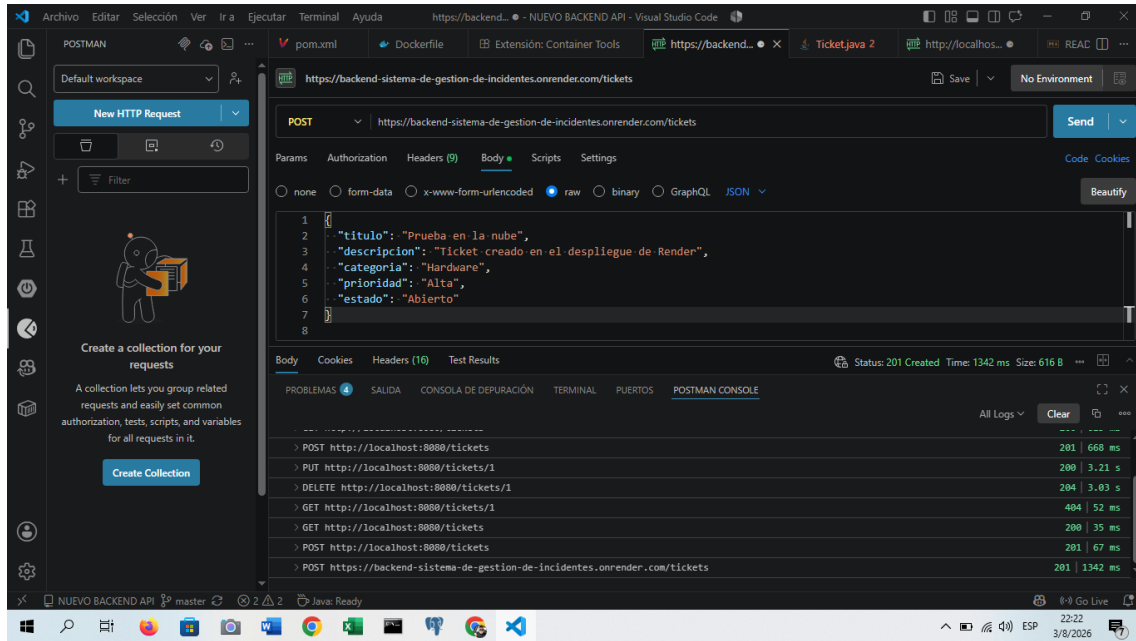


DELETE



DESPLIEGUE EN RENDEL







CONCLUSIONES

- ✓ Se cumplió con todos los requisitos establecidos en la guía de la Actividad 8.
- ✓ La API cuenta con una estructura organizada y fácil de mantener.
- ✓ La conexión con la base de datos PostgreSQL es estable y automática.
- ✓ Todas las operaciones de gestión de tickets funcionan correctamente y fueron validadas.
- ✓ El proyecto está listo para ser almacenado en GitHub y desplegado en Render sin inconvenientes.
- ✓ Se adquirió experiencia práctica en el desarrollo de soluciones backend integradas con bases de datos relacionales.

LINK DE GIT HUB

<https://github.com/lcongo1398/BACKEND-SISTEMA-DE-GESTION-DE-INCIDENTES.git>

LINK DESPLIEGE

<https://backend-sistema-de-gestion-de-incidentes.onrender.com/tickets>